```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>

#define NUM_DICE 5
#define NUM_CATEGORIES 13
#define BONUS_THRESHOLD 63
#define YAHTZEE_BONUS 50

typedef struct{
   int dice[NUM_DICE];
   int score[NUM_CATEGORIES];
   char name[50];
   int yahtzee_count; // track the number of yhatzee reroll
   int total_upper_score; // sum of scores from categories 1 to 6
   int bonus_added; //flag to indicate if the upper section bones has added
} Player;

const char *category_names[NUM_CATEGORIES] =
{
   "Ones","Twos","Threes","Fours","Five","Sixes","Three-of-a-Kind","Four-of-a-
Kind","Full House",
   "Small Straight","Large straight","Yahtzee","Chance"
};

//function prototype
void roll_dice(int dice[NUM_DICE]);
void display_dice(int dice[NUM_DICE]);
void count_dice(int dice[NUM_DICE], int counts[6]);
void display_scoreboard(Player* player);
int count_specific_number(int dice[NUM_DICE], int number);
int three_of_a_kind(int dice[NUM_DICE]);
int four_of_a_kind(int dice[NUM_DICE]);
int full_house(int dice[NUM_DICE]);
int small_straight(int dice[NUM_DICE]);
int large_straight(int dice[NUM_DICE]);
int yahtzee(int dice[NUM_DICE]);
int chance(int dice[NUM_DICE]);
void handle_reroll(Player *player);
int choose_category(Player *Player);
void ai_reroll_dice(int dice[NUM_DICE],int category);
int choose_ai_category(Player *Player);
```

```
void calculate_winner(Player players[2]);

//roll dices
void roll_dice(int dice[NUM_DICE])
{
   for (int i=0; i < NUM_DICE; i++)
   {
    dice[i] =rand() % 6 + 1;
   }
}
//display dices
void display_dice(int dice[NUM_DICE])
{
   printf("Dice:");
   for (int i=0; i<NUM_DICE; i++)
   {
      printf("%d ",dice[i]);
   }
   printf("\n");
}
//count of dic values
void count_dice(int dice[NUM_DICE], int counts[6])
{
   memset(counts, 0,sizeof(int)* 6);
   for (int i=0; i<NUM_DICE; i++)
   {
     counts[dice[i] - 1]++;
   }
}
//display score bord
void display_scoreboard(Player* player)
{
    printf("\n%s's Scoreboard:\n",player->name);
    for(int i=0 ;i<NUM_CATEGORIES ;i++){
       if (player->score[i] == -1)
       printf("%-15s: Not scored\n", category_names[i]);
       else
       printf("%-15s: %d\n",category_names[i],player->score[i]);
   }

printf("\n");

}
```

```
//count of specific number
int count_specific_number(int dice[NUM_DICE], int number)
{
   int count = 0;
   for (int i=0; i<NUM_DICE; i++)
   {
      if(dice[i]== number){
         count += number;
      }
   }
   return count;

}
//scoring functions

//three of kind
int three_of_a_kind(int dice[NUM_DICE])
{
   int counts[6];
   count_dice(dice,counts);
   for (int i=0; i<6; i++)
   {
      if (counts[i]>=3)
      {
         int total =0;
         for (int j=0; j<NUM_DICE; j++)
         {
            total += dice[j];
         }
         return total; //sum of all dice for three of kind
      }

   }
   return 0;
}

//four of a kind
int four_of_a_kind(int dice[NUM_DICE])
{
   int counts[6];
   count_dice(dice,counts);
   for (int i=0; i<6 ; i++)
   {
```

```
      if (counts[i]>=4)
      {
         int total = 0;
         for (int j=0; j<NUM_DICE; j++)
         {
            total += dice[j];
         }
         return total; // sum of all dice for four of kind

      }

   }
   return 0;
}

//full house
int full_house(int dice[NUM_DICE])
{
   int counts[6];
   count_dice(dice,counts);
   int has_three= 0 ,has_two = 0;
   for (int i=0; i<6 ; i++)
   {
      if(counts[i] == 3) has_three=1;
      if(counts[i] == 2) has_two=1;

   }
   return(has_three && has_two)? 25 :0; //25 points for full house

}
// small straight
int small_straight(int dice[NUM_DICE])
{
   int counts[6];
   count_dice(dice,counts);

  //check for 4 consecutive numbers
  if ((counts[0] && counts[1] && counts[2] && counts[3])  ||
     (counts[1] && counts[2] && counts[3] && counts[4])  ||
     (counts[2] && counts[3] && counts[4] && counts[5]))
  {
  return 30; //30 point for small straight
  }
```

```
   return 0;
}

//large straight
int large_straight(int dice[NUM_DICE])
{
   int counts[6];
   count_dice(dice,counts);

   //check for 5 consecutive numbers
   if ((counts[0] && counts[1] && counts[2] && counts[3] && counts[4]) ||
      (counts[1] && counts[2] && counts[3] && counts[4] && counts[5]))
      {
      return 40; // 40 points for large straight
   }
   return 0;
}
//yahtzee
int yahtzee(int dice[NUM_DICE])
{
    int counts[6];
   count_dice(dice,counts);
   for (int i=0; i<6 ; i++)
   {
      if(counts[i]==5){
         return 50; //50 point for yahtzee
      }
   }

}
//chance
int chance(int dice[NUM_DICE])
{
   int total =0;
   for (int i=0; i<NUM_DICE ; i++)
   {
      total += dice[i];
   }
   return total; //sum of all dice

}
//rerolling dice (human player)
void handle_reroll(Player *player)
```

```
{
   char reroll_choice;
   int reroll[NUM_DICE];
   for(int reroll_count= 0; reroll_count<2; reroll_count++)
   {
     printf("Do you want to reroll? (y/n): ");
      scanf(" %c", &reroll_choice);

     if(reroll_choice == 'y') //( y == yes and n == no)
     {
      int reroll[NUM_DICE] ={0};
       printf("Enter the dice numbers to reroll (1-5): "); // (1 for reroll wanted dices  || 0 for
keep dices )
          for(int i=0; i<NUM_DICE; i++)
          {
             reroll[i] = 0;
             printf("Dice %d: ",i + 1);
             scanf("%d",&reroll[i]);
          }
        for(int i=0; i<NUM_DICE; i++)
          {
             if (reroll[i] == 1) {
                player->dice[i] = rand()%6 + 1;
             }
          }
          display_dice(player->dice);
      }
      else {
          break;
     }

   }

}

//player's choice chtegory
int choose_category(Player *Player)
{
      int category;
      while (1)
      {
       printf("Choose a category:\n  0) Ones\n  1) Twos\n  2) Threes\n  3) Fours\n  4)
Fives\n  "
```

```
            "5) Sixes\n  6) Three-of-a-Kind\n  7) Four-of-a-Kind\n  8) Full House\n  "
            "9) Small Straight\n  10) Large Straigh\n  11)  Yahtzee\n  12) Chance:");
      scanf("%d",&category);
      if (category >= 0 && category < NUM_CATEGORIES && Player->score[category]
== -1) {
         break;
      } else {
         printf("Invalid choice or category already used. Please choose again.\n");
      }
   }
   return category;
      }

// AI Reroll option
void ai_reroll_dice(int dice[NUM_DICE],int category)
{
   int keep[NUM_DICE]={0}; // array to decide which dice to keep

   int counts[6];
   count_dice(dice, counts);
   if(category >= 0 && category <= 5) // ones to sixs (upper section)
   {
      int target = category +1;
      for(int i=0; i<NUM_DICE; i++)
      {
         if(dice[i] == target)
         {
            keep[i] = 1; // keep dice matching the target number
         }

      }

   } else if(category == 6 || category == 7) // three of kind and four of kind
   {
      int most_frequent = -1 , max_count = 0;
      for(int i = 0; i < 6; i++)
      {
         if(counts[i] > max_count)
         {
            most_frequent = i+1;
            max_count = counts[i];
         }
```

```
      }
      for (int i =0; i <NUM_DICE; i++)
      {
         if (dice[i] == most_frequent)
         {
            keep[i]= 1;

         }
      }
   } else if (category == 8) // full house
   {
    int most_freq1 = -1, most_freq2 = -1;
    for (int i= 0; i <6; i++)
    {
        if (counts[i] >= 2)
        {
         if (most_freq1 == -1 || counts[i]>counts[most_freq1 - 1])
         {
            most_freq2 = most_freq1;
            most_freq1 = i+1;
         } else if (most_freq2 == -1 || counts[i]> counts[most_freq2 -1])
         {
            most_freq2 = i +1;
         }


        }

    }
    for (int i = 0; i <NUM_DICE; i++)
    {
      if (dice[i] == most_freq1 || dice[i] == most_freq2)
      {
         keep[i] = 1;
      }
    }
} else if (category == 9 || category == 10) // small straight or large straight
{
   int consecutive = 0, start = 0;
   for (int i = 0; i < 6; i++)
   {
      if (counts[i] > 0)
      {
```

```
            consecutive++;
            if (consecutive == 4 && category == 9) break;
            if (consecutive == 5 && category == 10) break;

        } else {
            consecutive = 0;
        }
        if(consecutive >= 3) start = i - consecutive +2;
    }
    for (int i = 0; i < NUM_DICE; i++)
    {
        if (dice[i] >= start && dice[i] < start + 4)
        {
            keep[i] = 1;
        }

    }

}else if (category == 11) //Yahtzee
{
    int max_num = -1, max_count = 0;
    for (int i = 0; i < 6 ; i++)
    {
        if (counts[i]> max_count)
        {
            max_num = i + 1;
            max_count = counts[i];
        }
    }
    for (int i = 0; i < NUM_DICE; i++)
    {
        if (dice[i] >= 4)
        {
            keep[i] = 1;
        }
    }
}else if (category == 12) // chance
{
    for (int i = 0; i < NUM_DICE; i++)
    {
        if (dice[i] >= 4) // keep dice with value of 4 or more
        {
            keep[i] = 1;
```

```
      }

   }

}
// reroll only the dice not kept
for (int i = 0; i < NUM_DICE; i++)
{
   if (!keep[i])
   {
      dice[i] = rand() % 6 + 1;
   }

}
printf("AI rerolls dice for category %s:\n",category_names[category]);
display_dice(dice);
}

// AI choose category
int choose_ai_category(Player *Player){
   int best_category = -1;
   int best_score = 0;

   for (int category=0; category<NUM_CATEGORIES; category++)
   {
      if (Player->score[category] == -1)
      {
        int score =0;
        if (category >=0 && category <= 5)
        {
         score = count_specific_number(Player->dice, category +1);

        }else if (category == 6)
        {
         score = three_of_a_kind(Player->dice);

        }else if (category == 7)
        {
           score = four_of_a_kind(Player->dice);

        }else if (category == 8)
        {
           score = full_house(Player->dice);
```

```
        }else if (category == 9)
        {
          score = small_straight(Player->dice);

        }else if (category == 10)
        {
           score = large_straight(Player->dice);

        }else if (category == 11)
        {
           score = yahtzee(Player->dice);

        }else if (category == 12)
        {
          score = chance(Player->dice);
        }

        //choose the category with the highest potential score
        if (score > best_score)
        {
           best_score = score;
           best_category = category;
        }

      }

   }

   return best_category; //return the best category

}

//calculate and display the final score and winner
void calculate_winner(Player players[2])
{
   int score1 = 0, score2 =0;

   //calculate both player scores
   for (int i=0; i<NUM_CATEGORIES; i++)
   {
      score1 += players[0].score[i];
      score2 += players[1].score[i];
```

```
    }

 //uper section bonus is already include in score calculation
     printf("Final Score:\n%s: %d\n%s: %d\n" ,
players[0].name,score1,players[1].name,score2);
    if (score1 > score2)
    {
       printf("%s Wins!\n",players[0].name);
    }
    else if (score2 > score1)
    {
        printf("%s Wins!\n",players[1].name);
    }
   else
   {
    printf("It's a tie!\n");
   }

}
// Main game loop
int main() {
    printf("Welcome to Yahtzee!\n");

    Player players[2];
    strcpy(players[0].name, "Player 1");
    strcpy(players[1].name, "AI Player");
    srand(time(NULL));

    // Initialize players
    for (int p=0; p<2; p++)
    {
       for (int i=0; i<NUM_CATEGORIES; i++)
        {
           players[p].score[i] = -1;
       }
       players[p].yahtzee_count = 0;
       players[p].total_upper_score = 0;
       players[p].bonus_added = 0;
    }

    // Game loop: 13 rounds for both players
    for (int round =0; round<NUM_CATEGORIES; round++)
    {
```

```
    for (int i = 0; i < 2; i++)
    {
        printf("%s's turn (Round %d):\n", players[i].name, round + 1);

        // Roll dice and display
        roll_dice(players[i].dice);
        display_dice(players[i].dice);

        // Handle rerolling
        if (i == 0) { // Human player
            handle_reroll(&players[i]);
        } else { // AI player
            int chosen_category = choose_ai_category(&players[i]);
            printf("AI initially selected category: %d (%s)\n", chosen_category,
category_names[chosen_category]);

            // AI rerolls up to 2 times to optimize for the chosen category
            for (int reroll_count = 0; reroll_count < 2; reroll_count++) {
                ai_reroll_dice(players[i].dice, chosen_category);
            }
            display_dice(players[i].dice);
        }


        // Choose category
        int category;
        if(i == 0)
        {
            category = choose_category(&players[i]);
        } else
        {
            category = choose_ai_category(&players[i]);
            printf("AI conformed category: %d (%s)\n",
category,category_names[category]);
        }

        // Score based on chosen category
        switch(category)
        {
            case 0: case 1: case 2: case 3: case 4: case 5:
                players[i].score[category] = count_specific_number(players[i].dice, category +
1);
                players[i].total_upper_score += players[i].score[category];
```

```
            //apply upper section bonus only once if the threshold is reached
            if(players[i].total_upper_score > 63 && !players[i].bonus_added){
               players[i].score[12] += 35; //35 point bonus
               players[i].bonus_added = 1; //mark bonus as added
            }
            break;
         case 6:
            players[i].score[category] = three_of_a_kind(players[i].dice);
            break;
         case 7:
            players[i].score[category] = four_of_a_kind(players[i].dice);
            break;
         case 8:
            players[i].score[category] = full_house(players[i].dice);
            break;
         case 9:
            players[i].score[category] = small_straight(players[i].dice);
            break;
         case 10:
            players[i].score[category] = large_straight(players[i].dice);
            break;
         case 11:
            players[i].score[category] = yahtzee(players[i].dice);
            break;
         case 12:
            players[i].score[category] = chance(players[i].dice);
            break;
      }

      printf("%s scored %d points in category %d.\n\n", players[i].name,
players[i].score[category], category);
      if(i == 0){
      display_scoreboard(&players[i]);
    }
    }
    }


  // Calculate the winner
  calculate_winner(players);

    return 0;  }
```